Synopsis

Intelligent Network Intrusion Detection Using KNN and Feature Scaling for Enhanced Cyber Security Analysis.

Problem Id: 09

Team Id: PY-GT-58

Student Name: Sambhav Jaiswal (Team Leader)

University Roll number: 2028835

Section: G(G-2)

Student Name: Pragun Khanna

University Roll number: 2028798

Section: G(G-2)

Student Name: Utsav Bajoria

University Roll number: 2208896

Section: G(G-2)

- # Problem Description:

In the modern digital world, cybersecurity is a major concern due to the rapid increase in cyber-attacks. Networks are constantly exposed to threats such as hacking, malware, and unauthorized access. Detecting these threats manually is difficult because of the large amount of data generated in network traffic.

This project aims to build a basic Intrusion Detection System (IDS) using Machine Learning techniques. The system will analyse network data based on features like Packet Size, Connection Duration, and Error Rate. These features help in identifying patterns that indicate whether the traffic is normal or suspicious.

The K-Nearest Neighbours (KNN) algorithm will be used for classification. It is simple and suitable for beginners as it classifies data based on similarity. The Elbow Method will be used to find the optimal value of K, which improves the model's accuracy. This project helps first-year students understand how machine learning can be applied in real-world cybersecurity problems.

- # Steps of Implementation:

1. Dataset Creation:

A network log dataset is created or simulated containing features such as **Packet_Size, Connection_Duration, Error_Rate**, and labels indicating normal or malicious traffic.

2. Data Preprocessing

• Handling missing or inconsistent values in the dataset
• Normalizing numerical features for uniform scale
• Encoding labels (Normal = 0, Intrusion = 1)

3. Feature Scaling

Data is standardized using **StandardScaler** to ensure all features contribute equally to distance calculation in KNN.

4. Optimal K Selection (Elbow Method)

The Elbow Method is applied to determine the best value of **K (number of neighbors)** by analyzing error rates for different K values.

5. Train-Test Split

The dataset is divided into training and testing sets (80% training, 20% testing).

6. Model Training

A K-Nearest Neighbors (KNN) Classifier is trained on the scaled training data.

7. Model Evaluation

The model is evaluated using performance metrics such as:
• Accuracy
• Precision
• Recall
• Confusion Matrix

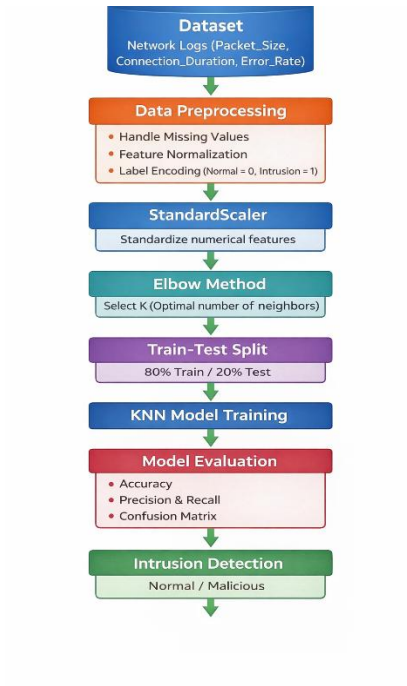

Fig. 1 Sample architecture

- **Libraries used :**

  NumPy: Used for numerical operations.
  Pandas: Used for handling datasets.
  Matplotlib: Used for plotting graphs.
  Scikit-learn: Used for ML algorithms and preprocessing.

- **Evaluation metrics and Plots used:**

  Accuracy: Measures correct predictions.
  Confusion matrix: Shows correct and incorrect classifications.
  Elbow Curve: Helps in selecting optimal K value.

- **Required topics from the subject:**

  Strings: Used for handling and manipulating textual data such as product reviews.
  Files: Used to read and store datasets in CSV format.
  Functions: Used to modularize code such as text cleaning and preprocessing functions.
  Loops: Used for iterating over data during preprocessing and training.

- **Platform Required:**

  Google Colab / Jupyter Notebook / Python IDE

- **Books and Link Sources:**

1.“Python for Data Analysis” by Wes McKinney

2. GeeksforGeeks (https://www.geeksforgeeks.org)

3. Scikit-learn official documentation

4. Kaggle datasets (https://www.kaggle.com)